CONFIGURATION MANUAL

XPLAINTM: EXPLAINABLE COST-SENSITIVE MACHINE LEARNING FOR AML TRANSACTION MONITORING

MSC FINTECH (MSCFTD1)

RESEARCH IN COMPUTING – PRACTICUM PART 2

DR. BRIAN BYRNE

Mohd Nizam Mohd Nasir Shaikh

Student ID: 24198170

Student Outlook: X24198170@student.ncirl.ie

Academic Integrity Code: SfYu3HrPhR

School of Computing

National College of IRELAND

National College of Ireland

Project Submission Sheet

Student Name:	Mohd Nizam Mohd Nasir Shaikh	
Student ID:	24198170	
Programme:	Master of Science in FinTech (MSCFTD1)	**Year:** 2026–2027
Module:	Research in Computing – Practicum Part 2	
Lecturer:	Dr. Brian Byrne	
Submission Due Date:	10th of August 2026	
Academic Integrity Code:	SfYu3HrPhR	

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard supplied by the library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Signature: Mohd Nizam **Date:** 10/08/2026

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
N/A	N/A	N/A

Academic Honesty Declaration

I declare the following to be true for this submission:

- I have completed the task during the designated time window and declare it to be exclusively my own work.
- I have not received, or attempted to receive assistance in preparing this response from any other person during the assessment window.
- I have not provided, or offered to provide, assistance to any other student by any means during the assessment window.
- I have read and understand the National College of Ireland guidelines of Plagiarism.

Table of Contents

1 System Requirements **1**
- 1.1 Hardware . . . 1
- 1.2 Software . . . 1
- 1.3 Dataset . . . 2
- 1.4 Python Libraries . . . 2

2 Environment Setup **3**
- 2.1 Runtime Configuration . . . 3
- 2.2 Getting to the Data . . . 3
- 2.3 Where the Outputs Go . . . 3

3 Configuration Parameters **4**
- 3.1 The DATASET_MODE Switch . . . 4
- 3.2 Everything Else . . . 5

4 Running the Pipeline **5**
- 4.1 Stage 1 — Ingestion and Audit . . . 5
- 4.2 Stage 2 — Behavioural Features . . . 6
- 4.3 Stage 3 — Temporal Split . . . 6
- 4.4 Stage 4 — Modelling and Evaluation . . . 7
- 4.5 Stage 5 — Explainability and Export . . . 8

5 Running All Three Regimes **8**
- 5.1 Order of Execution . . . 8
- 5.2 Surviving a Disconnect . . . 9

6 Outputs and Runtimes **10**
- 6.1 What Each Run Produces . . . 10
- 6.2 How Long It Takes . . . 10

7 Troubleshooting **11**

8 Source Code and Reproducibility **11**

References **12**

Configuration Manual

XplainTM — Explainable Cost-Sensitive Machine Learning for AML

Purpose of this Manual

This manual explains how to set up and run the artefact I built for my practicum, so that somebody else can reproduce the results I report in the accompanying research paper. The artefact is a single Jupyter notebook, `Practicum 2 - Artefact - Nizam Shaikh - 24198170.ipynb`. It is not three notebooks and it is not a package — everything lives in one file, and the whole result set is regenerated by changing one variable and running the cells from top to bottom.

I ran the notebook three times in total, once for each of the dataset regimes described in Section 3 of the research paper. What follows is written in the order I would actually do it if I were starting again on a fresh machine: check the hardware, install the libraries, attach the data, set the mode, run, and collect the outputs. I have included the problems I hit along the way, because most of them took longer to diagnose than to fix.

1 System Requirements

1.1 Hardware

I developed and ran the artefact entirely in the cloud, on Google Colab and Kaggle Notebooks, so no local installation is needed. That said, the free tiers of both platforms are close enough to their limits on the largest dataset that the hardware is worth stating properly.

- **Processor.** Any modern multi-core CPU is adequate. Every model except TabNet trains on CPU in a reasonable time.
- **GPU.** An NVIDIA T4 or equivalent. This is optional but I would recommend it: TabNet took roughly 925 seconds on HI-Large without one, and under 300 seconds with one. The other seven models barely notice.
- **RAM.** 12 GB minimum. Colab's free tier gives around 12 GB and Kaggle gives 16 GB. Both are sufficient, but only because of the row caps described in Section 3.2 — without them the largest run will not survive.
- **Storage.** At least 25 GB free. The raw HI-Large CSV alone is 15.88 GB, and the Parquet splits plus saved model artefacts add another 3 to 5 GB on top.

1.2 Software

The environment is a Linux backend, which is what both Colab and Kaggle provide by default. I used Python 3.12 on Colab, although anything from 3.10 upwards should work. The notebook checks which platform it is running on and picks the correct data path itself, so the same file works unmodified in either place.

One point that is easy to miss: internet access has to be switched on. On Colab it is on by default, but Kaggle disables it unless you turn it on under *Session options*. Two of the libraries I use — CatBoost and pytorch-tabnet — are not pre-installed on Kaggle, so with the internet off the very first cell fails with a name resolution error and nothing else runs.

1.3 Dataset

The data is the IBM Transactions for Anti-Money Laundering dataset published on Kaggle, generated by the AMLworld simulator (Altman et al., 2023). The Kaggle accession identifier is:

```
ealtman2019/ibm-transactions-for-anti-money-laundering-aml
```

Only the HI (High Illicit) dataset is used; the LI variants are not needed.

Table 1: The three dataset files required, one per regime

File	Regime	Size	Laundering rows
`HI-Small_Trans.csv`	small (1:10)	0.6 GB	5,177
`HI-Medium_Trans.csv`	medium (1:5)	3.5 GB	35,230
`HI-Large_Trans.csv`	large (1:1)	15.88 GB	225,546

Each file has eleven columns:

```
Timestamp|From Bank|Account|To Bank|Account|Amount Received|Receiving Currency
|Amount Paid|Payment Currency|Payment Format|Is Laundering
```

The repetition there is not a typo in this manual — the raw CSV genuinely names the sender and receiver columns identically. If you read it with pandas the second one is silently renamed and you can end up joining on the wrong field without noticing. I handle it by passing explicit replacement names at scan time, so the two become `From Account` and `To Account`. This is covered in Section 4.1.

1.4 Python Libraries

Everything installs from a single cell at the top of the notebook.

```
!pip install -q polars pyarrow kagglehub xgboost imbalanced-learn \
              shap lightgbm catboost pytorch-tabnet statsmodels
```

The imports are grouped by purpose in the second cell:

```
import polars as pl
import pandas as pd, numpy as np
import os, gc, json, time, warnings
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.metrics import (roc_auc_score, average_precision_score,
                             accuracy_score, confusion_matrix,
                             classification_report, precision_recall_curve)
```

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from pytorch_tabnet.tab_model import TabNetClassifier
from statsmodels.stats.contingency_tables import mcnemar
import shap
import kagglehub
```

2 Environment Setup

2.1 Runtime Configuration

On Google Colab. Open the notebook, go to *Runtime* → *Change runtime type*, set the hardware accelerator to GPU and pick T4, then save and let the runtime restart.

On Kaggle. Create a new notebook and use *File* → *Import Notebook* to upload the `.ipynb`. In the panel on the right, click *Add Data* and search for "IBM Transactions for Anti Money Laundering"; the dataset by `ealtman2019` is the correct one. Still in that panel, open *Session options*, set the accelerator to GPU T4 x2, and switch *Internet* to On. Save and the session restarts with everything in place.

Of the two, I found Kaggle the more reliable environment for the HI-Large run. Colab's free tier disconnects after about 90 minutes of browser inactivity and it wipes the local filesystem when it does. Kaggle allows much longer sessions and commits the output folder to persistent storage when you click *Save Version*.

2.2 Getting to the Data

I did not want to maintain two versions of the notebook, so the data access block works out where it is running and behaves accordingly. On Kaggle the dataset is already mounted read-only and is read in place. On Colab there is nothing mounted, so `kagglehub` downloads it once and caches it for later sessions.

```
import os, kagglehub

KAGGLE_PATH = '/kaggle/input/ibm-transactions-for-anti-money-laundering-aml'

if os.path.exists(KAGGLE_PATH):
    BASE = KAGGLE_PATH
    print('Kaggle environment detected - reading dataset in place.')
else:
    BASE = kagglehub.dataset_download(
        'ealtman2019/ibm-transactions-for-anti-money-laundering-aml')
    print('Colab environment - dataset cached at:', BASE)

FILE_PATH = os.path.join(BASE, DATASET_FILE)
```

2.3 Where the Outputs Go

Each run writes into its own subdirectory, named after the regime. That was a deliberate choice: the first time I re-ran the notebook in a different mode I overwrote the previous run's charts, and had to run it again to get them

back.

```
BASE_OUT = '/kaggle/working/xplaintm_outputs' if os.path.exists('/kaggle/working') \
           else '/content/xplaintm_outputs'
OUT_DIR  = os.path.join(BASE_OUT, DATASET_MODE)
os.makedirs(OUT_DIR, exist_ok=True)
```

3 Configuration Parameters

3.1 The DATASET_MODE Switch

This is the single most important line in the notebook. Changing it and re-running all cells produces the complete set of results for that regime — the metrics tables, the charts, the SHAP plots, the alert export and the JSON log.

I built it this way because my supervisor asked for results across all three dataset scales and the cleanest way to make those results comparable was to put them through an identical code path rather than maintaining three separate notebooks that would inevitably drift apart.

```
# ---- SINGLE POINT OF CONTROL ----
DATASET_MODE = 'large'          # 'small' | 'medium' | 'large'

MODE_CONFIG = {
    'small' : dict(file='HI-Small_Trans.csv',  neg_per_pos=10),
    'medium': dict(file='HI-Medium_Trans.csv', neg_per_pos=5),
    'large' : dict(file='HI-Large_Trans.csv',  neg_per_pos=1),
}

DATASET_FILE = MODE_CONFIG[DATASET_MODE]['file']
NEG_PER_POS  = MODE_CONFIG[DATASET_MODE]['neg_per_pos']
```

`NEG_PER_POS` sets how many legitimate transactions are kept for every laundering transaction. Every laundering row is retained in all three regimes — only the majority class is sampled down. A value of 10 gives the 1:10 Small regime, 5 gives 1:5 Medium, and 1 gives the balanced 1:1 Large regime.

One thing worth flagging for anyone modifying this: the sampling has to happen at ingestion, before the rolling features are computed. An earlier version of my notebook applied it later, which quietly produced a 50:50 dataset regardless of what `DATASET_MODE` was set to, because the ratio logic was being applied to data that had already been balanced upstream.

3.2 Everything Else

Table 2: The remaining configuration constants and what they control

Constant	Value	What it does
RANDOM_STATE	42	Seed passed to every stochastic operation
TARGET_COL	Is Laundering	Name of the binary target column
TRAIN_FRAC, VAL_FRAC	0.70, 0.15	Temporal split points; test is the remainder
REPORT_THRESHOLD	10,000	Reporting threshold used by the structuring flag
SMOTE_MAX_ROWS	300,000	Row cap on the SMOTE training arm
TABNET_MAX_ROWS	300,000	Row cap on TabNet training
SHAP_SAMPLE	2,000	Test rows sampled for the TreeSHAP explainer
ALERT_EXPORT_MAX	1,000	Rows written to the STR decision-support CSV
TUNE_ITER, TUNE_CV	10, 3	RandomizedSearchCV iterations and folds

The three row caps are what keep the notebook inside Colab's memory limit at HI-Large scale. They apply only to the SMOTE and TabNet training arms and to the SHAP explainer — all of the reported evaluation metrics are still computed on the complete test set, so the caps affect runtime rather than results.

4 Running the Pipeline

The notebook is laid out in five stages. Running every cell top to bottom executes the full pipeline for whichever mode is currently selected. What follows describes each stage and shows the part of the code that does the actual work.

4.1 Stage 1 — Ingestion and Audit

Polars scans the CSV lazily rather than reading it into memory. That matters at HI-Large scale: at 15.88 GB the file is larger than the RAM available on either platform, so a lazy scan builds a query plan first and materialises only the rows that survive the filters. The class-ratio sampling is applied here, before anything else touches the data.

```
lf = pl.scan_csv(FILE_PATH, has_header=True,
                 new_columns=RAW_COLUMNS,      # resolves duplicate 'Account'
                 schema_overrides={...})

lf_valid = lf.filter(pl.col('Amount Paid') >= 0)

pos_df = lf_valid.filter(pl.col(TARGET_COL) == 1).collect()
n_neg  = pos_df.height * NEG_PER_POS
neg_df = lf_valid.filter(pl.col(TARGET_COL) == 0).collect() \
                 .sample(n=n_neg, seed=RANDOM_STATE)

df = pl.concat([pos_df, neg_df])
```

The audit output that follows reports null counts, exact duplicate counts, the laundering prevalence and the time span of the data. On all three files it came back clean: no nulls in any field once the schema was enforced and no duplicate rows.

4.2 Stage 2 — Behavioural Features

This is the stage I would defend most strongly if asked. All of the rolling behavioural features are built with Polars window functions, partitioned by account and ordered strictly by timestamp.

That combination is what makes them causal. A window anchored to `Timestamp` can only see rows at or before the current one, so it is not possible for a future transaction to influence a feature. The guarantee comes from the shape of the operation rather than from remembering to be careful, which is why I chose to implement it this way.

```
df = df.sort(['from_id','Timestamp']).with_columns(
    pl.col('ones').rolling_sum_by('Timestamp', window_size='1h')
                  .over('from_id').alias('sent_count_1h'),
    pl.col('ones').rolling_sum_by('Timestamp', window_size='24h')
                  .over('from_id').alias('sent_count_24h'),
    pl.col('Amount Paid').rolling_sum_by('Timestamp', window_size='24h')
                  .over('from_id').alias('sent_sum_24h'),
    pl.col('Timestamp').diff().over('from_id').dt.total_seconds()
                  .fill_null(-1).alias('secs_since_last_sent'),
)

# Expanding unique-counterparty counts (fan-out / fan-in)
df = df.sort('Timestamp').with_columns(
    pl.struct(['from_id','to_id']).is_first_distinct().alias('first_pair')
).with_columns(
    pl.col('first_pair').cum_sum().over('from_id').alias('fan_out_unique_todate'),
    pl.col('first_pair').cum_sum().over('to_id').alias('fan_in_unique_todate'),
)
```

The last two features are the ones that turned out to matter most. They count how many distinct counterparties an account has sent to or received from up to that point, which is a tabular stand-in for the fan-in and fan-out subgraph patterns that Egressy et al. (2024) identify as the strongest signals in this dataset. When I looked at the SHAP output at the end of the pipeline, those two were the dominant drivers — so the approximation appears to be doing its job.

4.3 Stage 3 — Temporal Split

The data is sorted by timestamp and cut at the 70th and 85th percentiles. The splits are written to Parquet so that the modelling stage can be re-run without recomputing features, which saved a considerable amount of time during development.

Table 3: Split boundaries and row counts from the HI-Large run

Split	Boundary	Rows	Prevalence
Train	up to 2022-10-13 18:02	315,766	45.63%
Validation	up to 2022-10-28 07:50	67,663	52.33%
Test	after 2022-10-28 07:50	67,663	68.06%

The prevalence figures drift upwards across the three splits. That is the simulation evolving over time rather than an error, and it is the reason I report the majority-class baseline alongside every accuracy figure in the research paper.

4.4 Stage 4 — Modelling and Evaluation

Eight model configurations are trained in one loop, with the rules baseline evaluated separately using thresholds calibrated on the training set's 99th percentiles. Cost-sensitivity is supplied through `scale_pos_weight` for the tree models and through per-sample weights for TabNet, which does not expose that parameter.

```
spw = float((y_train == 0).sum() / max((y_train == 1).sum(), 1))

XGB_BASE = dict(n_estimators=400, max_depth=6, learning_rate=0.05,
                subsample=0.8, colsample_bytree=0.8,
                tree_method='hist', eval_metric='aucpr',
                n_jobs=-1, random_state=RANDOM_STATE)

models = {
  'Logistic Regression'     : Pipeline([('prep', prep_scaled),
                              ('clf', LogisticRegression(max_iter=2000,
                                      class_weight='balanced'))]),
  'Random Forest'           : Pipeline([('prep', prep_tree),
                              ('clf', RandomForestClassifier(n_estimators=200,
                                      max_depth=12,
                                      class_weight='balanced_subsample'))]),
  'LightGBM'                : Pipeline([('prep', prep_tree),
                              ('clf', LGBMClassifier(n_estimators=400,
                                      learning_rate=0.05, max_depth=6,
                                      scale_pos_weight=spw))]),
  'XGBoost (plain)'         : Pipeline([('prep', prep_tree),
                              ('clf', XGBClassifier(**XGB_BASE))]),
  'XGBoost (cost-sensitive)': Pipeline([('prep', prep_tree),
                              ('clf', XGBClassifier(**XGB_BASE,
                                      scale_pos_weight=spw))]),
  'XGBoost (SMOTE)'         : ImbPipeline([('prep', prep_tree),
                              ('smote', SMOTE(random_state=RANDOM_STATE)),
                              ('clf', XGBClassifier(**XGB_BASE))]),
  'CatBoost'                : CatBoostClassifier(iterations=400, depth=6,
                              learning_rate=0.05,
                              class_weights={0:1.0, 1:spw},
                              cat_features=cat_idx, verbose=0),
  'TabNet (neural network)' : TabNetClassifier(n_d=32, n_a=32, n_steps=3,
                              gamma=1.3, mask_type='sparsemax',
                              seed=RANDOM_STATE),
}
```

Wrapping each model in a scikit-learn `Pipeline` is not cosmetic. It means the preprocessor is fitted inside cross-validation, so scaling statistics computed on a validation fold never find their way into training.

The rules baseline thresholds, calibrated on the HI-Large training set, are: hourly velocity above 11, daily velocity above 210, fan-out above 8,365 unique counterparties, 24-hour outflow above USD 105,362,567.60 and a single-transaction amount above USD 13,613,316.92. A transaction is flagged if any one of the five is breached. Calibrating these on training data only is what keeps the baseline honest — tuning them on the test set would flatter the rules and understate the gap the research paper reports.

Tuning uses `RandomizedSearchCV` with 10 candidates over 3 folds, scored on average precision. The configuration it selected and the decision threshold that follows from it, are the values behind every HI-Large result in the paper:

```
Best params : subsample=0.8, scale_pos_weight=2.3830,
              n_estimators=800, min_child_weight=1,
              max_depth=6, learning_rate=0.05, colsample_bytree=0.7
Best CV PR-AUC : 0.9633

Validation-selected threshold : 0.5897
  val F1 = 0.917 | precision = 0.863 | recall = 0.978
Test accuracy at threshold    : 0.9422
  (majority-class baseline 0.6806 -> +26.2 points of lift)
```

4.5 Stage 5 — Explainability and Export

TreeSHAP is computed on a 2,000-row subsample of the test set, which keeps the runtime sensible without materially changing the ranking of the drivers. For each alerted transaction the three largest positive SHAP values are looked up in the reason-code dictionary and written into the export alongside the calibrated risk score.

```
explainer   = shap.TreeExplainer(final_clf)
shap_values = explainer(X_shap)

REASON_CODES = {
  'sent_count_1h'        : ('RC01','High transaction velocity within 1 hour'),
  'sent_count_24h'       : ('RC02','High transaction velocity within 24 hours'),
  'sent_sum_24h'         : ('RC03','Large cumulative outflow within 24 hours'),
  'near_threshold'       : ('RC04','Amount just below reporting threshold'),
  'fan_out_unique_todate': ('RC05','Funds dispersed to unusually many parties'),
  'fan_in_unique_todate' : ('RC06','Funds received from unusually many parties'),
  'recv_count_24h'       : ('RC07','High inbound transaction velocity'),
  'is_cross_currency'    : ('RC08','Cross-currency transfer'),
  'amount_log'           : ('RC09','Unusually large transaction amount'),
  'Payment Format'       : ('RC10','Higher-risk payment format or currency'),
  'secs_since_last_sent' : ('RC11','Unusual timing relative to account history'),
  'round_amount'         : ('RC12','Round-value or internal routing pattern'),
}
```

The file this produces, `str_decision_support.csv`, is the output that would matter operationally. It is a ranked list of the top 1,000 alerts with the transaction details, the calibrated score, the true label and the top three reason codes for each one — essentially what a compliance analyst would work from. Inference throughput for the tuned model measured 315,706 transactions per second, so scoring is not the bottleneck in any realistic deployment.

5 Running All Three Regimes

5.1 Order of Execution

The three runs are sequential, and the only thing that changes between them is `DATASET_MODE`. Each run writes a file called `results_log_<mode>.json` into `BASE_OUT` and the cross-dataset comparison section near the end of the notebook reads whichever of those three files exist and assembles the comparison table and chart automatically.

1. Set `DATASET_MODE = 'small'` and run all cells. Check that `results_log_small.json` has been written before moving on.

2. Set `DATASET_MODE = 'medium'` and run all cells.
3. Set `DATASET_MODE = 'large'` and run all cells. The cross-dataset section now has all three logs and produces the final comparison.

There is no need to restart the kernel between runs. The log files sit on disk independently of the Python session, so a run that is interrupted after the persistence cell has executed does not have to be repeated.

5.2 Surviving a Disconnect

Colab clears its local filesystem when a session ends, which I found out the hard way after losing a completed Medium run. The fix is to copy each log to Google Drive as soon as it is written, so that a timeout costs you the session but not the result.

```
from google.colab import drive
import shutil, os

drive.mount('/content/drive', force_remount=False)
drive_dir = '/content/drive/MyDrive/XplainTM'
os.makedirs(drive_dir, exist_ok=True)

log_path = os.path.join(BASE_OUT, f'results_log_{DATASET_MODE}.json')
shutil.copy(log_path, f'{drive_dir}/results_log_{DATASET_MODE}.json')
print(f'Backed up results_log_{DATASET_MODE}.json to Drive')
```

On Kaggle none of this is necessary. Clicking *Save Version* after each run commits everything in `/kaggle/working` to persistent storage, and it is still there when you open the notebook again.

6 Outputs and Runtimes

6.1 What Each Run Produces

Table 4: Files written to `OUT_DIR` by a single run

File	Contents
`aml_train / val / test.parquet`	The temporally split feature tables
`split_meta.json`	Split boundaries, row counts, prevalences
`model_comparison_validation.csv`	Validation metrics for every model
`model_comparison_results.csv`	Test metrics for every model
`results_log_<mode>.json`	Machine-readable log read by the cross-dataset cell
`xplaintm_final_model.joblib`	The tuned and calibrated model
`str_decision_support.csv`	Ranked alerts with scores and reason codes
`shap_summary.png`	TreeSHAP global feature importance
`tabnet_importance.png`	TabNet attention-based importance
`eval_confusion_matrix.png`	Confusion matrix at the selected threshold
`eval_pr_curve.png`	Precision-recall curve with the threshold marked
`calibration_curve.png`	Reliability diagram, before and after Platt scaling
`ks_lift_curve.png`	KS separation and lift chart
`model_comparison_prauc.png`	PR-AUC comparison bar chart
`model_comparison_recall.png`	Recall-at-capacity comparison chart

Two of the figures in the research paper — the temporal split diagram and the pipeline architecture diagram — are not produced by the notebook. I drew those separately from the values in `split_meta.json` and from the stage structure described in Section 4 of this manual.

6.2 How Long It Takes

The times below are from the HI-Large run on a Colab T4 session. The smaller regimes finish proportionally faster.

Table 5: Training time per model on HI-Large

Model	Training time	Validation PR-AUC
LightGBM	4 s	0.9719
XGBoost (plain)	5 s	0.9724
XGBoost (cost-sensitive)	5 s	0.9723
Logistic Regression	13 s	0.9390
XGBoost (SMOTE)	30 s	0.9723
Random Forest	40 s	0.9696
CatBoost	41 s	0.9723
TabNet (neural network)	925 s	0.9701

The gap between TabNet and the boosted trees is striking: 925 seconds against 4 seconds for LightGBM, for a validation PR-AUC that is marginally lower. TabNet stops early with a patience of 10 epochs, halting at epoch 38 with the best result at epoch 28, so it is not simply running longer than it needs to. End to end, a complete HI-Large run — ingestion of the 15.88 GB file, feature engineering, all eight models, tuning, SHAP and export — takes between 45 minutes and an hour on a T4.

7 Troubleshooting

Everything in this section is something I actually ran into. I have written down what the error looked like as well as what caused it, because the message is not always a good guide to the problem.

Table 6: Problems encountered and how to resolve them

What you see	What is wrong and how to fix it
`ModuleNotFoundError: pytorch_tabnet`, preceded by repeated name resolution warnings	Internet is switched off on Kaggle. Turn it on under *Session options* → *Internet*, re-run the install cell, then restart the session. The retry warnings are the giveaway — pip is not failing to find the package, it cannot reach PyPI at all.
`FileNotFoundError` on the CSV path	The dataset is not attached. On Kaggle use *Add Data*; on Colab confirm that `kagglehub` finished and print the resolved path. Running `os.walk('/kaggle/input')` lists exactly what is mounted.
Session crashes partway through HI-Large ingestion	Out of memory. Check that the lazy scan is being used rather than `pl.read_csv`, and that sampling is applied before feature engineering rather than after.
Prevalence reports 50% when the mode is set to small or medium	`NEG_PER_POS` is not reaching the sampling step. The negative sample must use `n = pos_df.height * NEG_PER_POS`, not `n = pos_df.height`. This is the bug I mentioned in Section 3.1.
The cross-dataset section reports missing logs	One or more regimes have not been run yet. The cell skips absent logs and prints which ones it could not find, so it is safe to run before all three are complete.
`SameFileError` during the Drive backup	Harmless — the log has already been copied. It happens when the file search includes the Drive directory and finds the copy it just made. Exclude that directory from the search path.

8 Source Code and Reproducibility

The artefact notebook, `Practicum 2 - Artefact - Nizam Shaikh - 24198170`, is submitted alongside this manual and the research paper. The dataset is publicly available on Kaggle (Altman et al., 2023) at:

https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml

There are no credentials or API keys anywhere in the notebook, and no hard-coded absolute paths outside the environment-detection block shown in Section 2.2. A third party should be able to run it on either platform without editing anything beyond `DATASET_MODE`.

On reproducibility: `RANDOM_STATE` is fixed at 42 and is passed to every stochastic operation in the pipeline — the majority-class sample, SMOTE, all model constructors, `RandomizedSearchCV` and the SHAP subsample. Re-running on the same platform reproduces the reported figures exactly. I did notice small differences of around 0.001 PR-AUC when comparing a CPU run against a GPU run, which comes from the order in which floating-point values are summed inside the XGBoost and LightGBM histogram builders. It is not a seeding problem and does not change any conclusion, but it is worth knowing about if your numbers differ in the fourth decimal place.

References

References

Altman, E., Blanuša, J., von Niederhäusern, L., Ott, J., Anagnostopoulos, E. and Atasu, K. (2023) 'Realistic synthetic financial transactions for anti-money laundering models', *Advances in Neural Information Processing Systems*, 36.

Arik, S.Ö. and Pfister, T. (2021) 'TabNet: Attentive interpretable tabular learning', *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8), pp. 6679–6687.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.

Egressy, B., von Niederhäusern, L., Blanuša, J., Altman, E., Wattenhofer, R. and Atasu, K. (2024) 'Provably powerful graph neural networks for directed multigraphs', *Proceedings of the AAAI Conference on Artificial Intelligence*, 38.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.Y. (2017) 'LightGBM: A highly efficient gradient boosting decision tree', *Advances in Neural Information Processing Systems*, 30, pp. 3146–3154.

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, 30, pp. 4765–4774.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V. and Gulin, A. (2018) 'CatBoost: Unbiased boosting with categorical features', *Advances in Neural Information Processing Systems*, 31, pp. 6638–6648.